Dynamics Queue Admin

System & Architecture Design Document — dispatch-queue management console (`queadmin-frontend`, SPA), version 1.0.0

Document	**SDD-1.0.0** — System & Architecture Design Document
Application	Dynamics Queue Admin (*Queue Admin*) — dispatch-queue management console of the Dynamics freight marketplace
Asset type	Web Application
Asset code	**ASSET-DYN-QA-001**
Standard	INSA Secure Website Management Standard, Version 1.0, 2014 EC
Platform	Vite 8 + React 19 + TypeScript SPA (React Router 7), Redux Toolkit / RTK Query, socket.io-client 4, hand-written CSS
Production host	`queue.dynamicsroute.tech`
Design version	1.0.0 (as-built)
Companion documents	SRS, Technical Specifications (TDD/LLD), User Documentation (User Guide), Security Documentation
Submitting entity	Dynamics Transport
Contact person	Marew Masresha, General Manager — `mmarew@gmail.com`, `0983222221`
Prepared by	Marew Masresha, General Manager
Reviewed by	Marew Masresha, General Manager
Signatories	Marew Masresha, General Manager
Date of preparation	15 October 2026
Date of submission	15 October 2026

As-built document

What follows is the design of the Dynamics Queue Admin console release we are submitting, as it actually exists in `queadmin-frontend`. It is the design side of the SRS and was written for review under the INSA Secure Website Management Standard. Where the backend (`transportBackEndNative`) is what really enforces a behaviour, we say so in the text, so a reviewer can tell what the client is responsible for and what the server decides.

Contents

1	Introduction
2	System overview and context
3	Architectural design
4	Functional decomposition
5	Component design
6	Data design
7	Interface design
8	Security design
9	Deployment and hosting
10	Cross-cutting design
11	Design decisions and trade-offs
12	Requirements-to-design traceability
13	Known limitations and future work

List of Figures

2.1	Level 0 — Context diagram (external ecosystem)
3.1	Level 2 — Architecture layers
4.1	Level 1 — Functional decomposition

1. Introduction

1.1 Purpose

This is the architecture and internal design of the Dynamics Queue Admin web console (`queadmin-frontend`, SPA), written against the code as we built it. It covers the system context, the layers, what the console breaks down into functionally, the components, the data it holds, the interfaces it calls, how the thing is secured, and how it is deployed. INSA reviews our submission against this document and the Secure Website Management Standard (Version 1.0, 2014 EC).

1.2 Scope

In scope is the client application only (Web Application, asset code **ASSET-DYN-QA-001**, version 1.0.0). The backend service (`transportBackEndNative`, REST + Socket.IO) appears here only where this SPA depends on its contracts; its internal design belongs to a separate document. The console runs a single kind of tenant, a *loading place* (a *Queue Organization*), and it does three things for that tenant: manage the loading place, run the FIFO driver queue, and handle bidding for shippers.

1.3 References

- `src/utils/constant.ts`, the single source of truth for REST endpoint paths
- `src/lib/redux/api/*.ts`, the endpoint modules and the RTK Query contract
- `src/lib/redux/api/base.ts`, the base query, credentials, 401 → logout
- `src/lib/socket.ts`, the Socket.IO client with its subscriptions and debounced invalidation
- `src/routes/AppRoutes.tsx`, the route table and lazy loading
- `src/components/auth/ProtectedRoute.tsx`, `src/components/auth/RoleGuard.tsx`
- `src/types/queue.ts` and `src/lib/redux/api/types.ts`, which hold the domain types
- `src/data/insaStandard.ts`, the INSA standard content behind the compliance dashboard
- `vite.config.ts` and `.env.example`, which describe the build and the environment model
- `docs/insa/assets/*.mmd` and `*.svg`, the sources for the figures

1.4 Definitions

Term	Definition
Queue Organization	A “loading place” tenant of the console (customs, factory, cement, depot, mine, farm, port, other) with coordinates, an optional check-in radius, and an approval status.
Queue Org Admin	Role id 11. Administers the loading place: profile, membership, and approval of bidding per order.
Queue Dispatcher	Role id 12. Runs the FIFO queue: manual check-in, dispatch, override, remove, history.
Admin / Super Admin	Role id 3 / 6. Approve loading places and oversee all organizations. Short session (24h).
FIFO queue	First-in-first-out driver waiting line per vehicle type, keyed by `queueNumber`; entries are dispatched, overridden, or removed.

Term	Definition
RTK Query	Server-state data fetching/caching layer; one base `api` extended by `injectEndpoints`.
Entry history / audit	Per-entry change log (column, old value, performed by, performed at) and the backend queue audit trail.

2. System overview and context

Dynamics Queue Admin is the dispatch-queue management console of the Dynamics freight marketplace. Its display name is “Queue Admin”, its asset type is Web Application, its asset code is **ASSET-DYN-QA-001**, and the version covered here is 1.0.0. Underneath it is a single-page React application served in production at `queue.dynamicsroute.tech`.

A Queue Organization is a physical loading place. Drivers who get there prove they are present with a GPS check-in and join a FIFO line. Dispatchers hand queued drivers out to shipper requests, and can move an entry up the order or drop it entirely as long as they give a reason that gets recorded. Queue-org admins look after the place’s profile and its members and decide, order by order, whether bidding is open. Platform admins approve origins and keep watch over every organization. All of those business rules are enforced by the backend (`transportBackEndNative`); the console is the layer that renders them and drives them through the REST and Socket.IO contracts.

2.1 External actors and systems

Actor/System	Role
Shipper	Creates freight requests (individual or batch); receives bid outcomes for dispatched drivers.
Driver / Vehicle Owner	Checks in with GPS, holds a place in the FIFO queue, bids, and updates journey status. Every driver is represented by a vehicle-driver assignment.
Queue Org Admin (11) / Queue Dispatcher (12)	The day-to-day operators of the console; they run the loading-place queue and the bidding.
Admin (3) / Super Admin (6)	Approve loading places and oversee organizations across the marketplace.
Backend REST API	`/api` endpoints consumed via RTK Query and legacy axios (`src/utils/constant.ts`).
Backend Socket.IO	Realtime queue fan-out; client subscribes with `queue:subscribe` (`src/lib/socket.ts`).
Photon geocoder (optional)	Place search for order origin/destination (`VITE_PHOTON_URL`, default `photon.komoot.io`). Falls back to a built-in list of Ethiopian cities.
OpenStreetMap / Nominatim / OSRM	Listed as optional external map services; not wired in this console asset (place search uses Photon). Where absent, no map tiles or routing calls are made.

2.2 Level 0 context

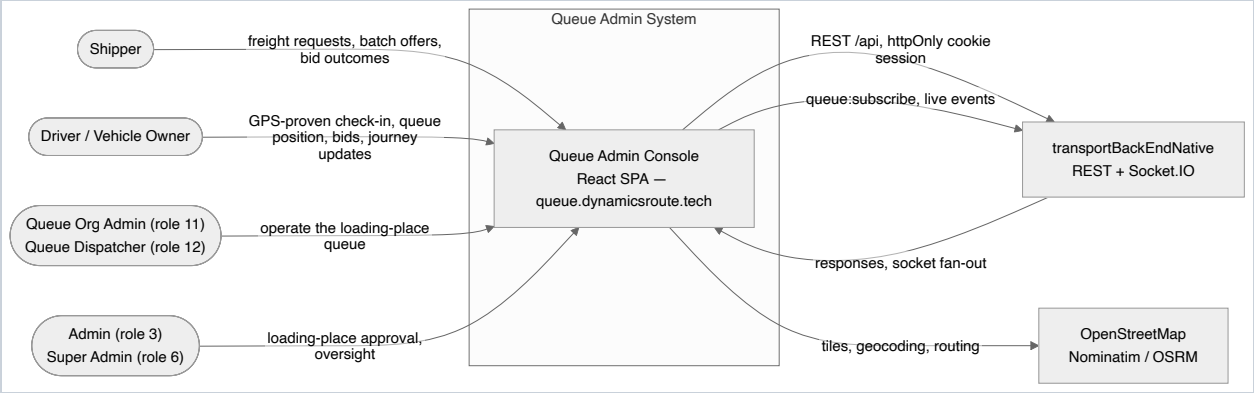


Figure 2.1. Level 0 — Context diagram (external ecosystem). Flows: freight requests / batch offers / bid outcomes, GPS-proven check-in and queue position, loading-place operation, approval and oversight, REST `/api` + cookie session, and Socket.IO events (per `assets/l0.mmd`).

3. Architectural design

3.1 Architectural goals

- A single-page React SPA where route authorization is layered and strict (`ProtectedRoute` → `RoleGuard`).
- API access goes through one place: a single RTK Query base `api` that feature endpoint modules extend, plus a legacy axios instance for the few services still using it.
- The session lives in a cookie, never in a token JavaScript can read, and one 401 handler logs the user out everywhere at once.
- Queue updates arrive over Socket.IO subscriptions, and the cache invalidation they set off is debounced so bursts settle into a single refetch.
- Presentation, application, state/data, API contract and external concerns each sit in their own layer.
- CSS is written by hand on top of design tokens, with no UI framework to carry around.

3.2 Layered model

Layer	Contents	Responsibility
Presentation	Pages and components under `src/pages/*` and `src/components/*` ; hand-written CSS (`src/index.css` tokens + co-located sheets; `src/dark.css`)	Render UI, queue board, orders, orgs, reports, settings, security; accessibility.
Application	`src/routes/AppRoutes.tsx` , guards, `src/context/AuthContext.tsx` , `src/contexts/ThemeContext.tsx` , composed hooks, forms (RHF + zod)	Routing, authorization, session lifecycle, form state and validation.
State & Data	RTK Query store (`src/lib/redux/store.ts`), slices (`authSlice` , `queueSlice`), zustand (`src/store/queueAdminStore.ts`), Socket.IO client, legacy axios services	Server-state cache, client state, realtime synchronization, HTTP transport.
API contract	`src/utils/constant.ts` and endpoint modules (`authEndpoints` , `queueOrganizationEndpoints` , `driverQueueEndpoints` , `shipperRequestEndpoints` , `vehicleEndpoints`)	Declare endpoints, methods, arguments, responses, and cache tags.
External	Backend REST `/api` , Socket.IO, optional Photon geocoder	Remote systems contracted through the API layer.

3.3 Technology selection

Technology	Version	Justification
Vite	8	Fast dev/build; env-driven base URL, dev proxy, dynamic-import route chunks, manual chunking.
React	19	SPA UI with concurrent rendering; StrictMode in `main.tsx` .
TypeScript	6	Type safety across store, endpoint contracts, components, and domain types.

Technology	Version	Justification
react-router-dom	7 (BrowserRouter)	Client routing, redirects, lazy routes, guarded access.
Redux Toolkit + RTK Query	2	Predictable client state plus server-state caching, tags, and invalidation.
React Context	—	Low-frequency global state: authentication (`AuthContext`) and theme.
zustand	5	Single persisted store for `selectedOrgId` and the socket-connected flag.
react-hook-form + zod	7 / 4	Form state and schema validation with input sanitization helpers.
axios	1	Shared instance for legacy services; `withCredentials` and 401 interceptor.
socket.io-client	4	Realtime queue events; subscriptions and debounced cache invalidation.
sonner	2	User-facing toast notifications for operations and errors.
date-fns	4	Date formatting and comparison for queue/order dates.
i18next / react-i18next	26 / 17	English + Amharic localization with inline fallback strings.
lucide-react	—	Consistent icon set used across the UI.
Hand-written CSS	—	Design tokens in `src/index.css` plus co-located sheets; no Tailwind.

3.4 Level 2 architecture layers

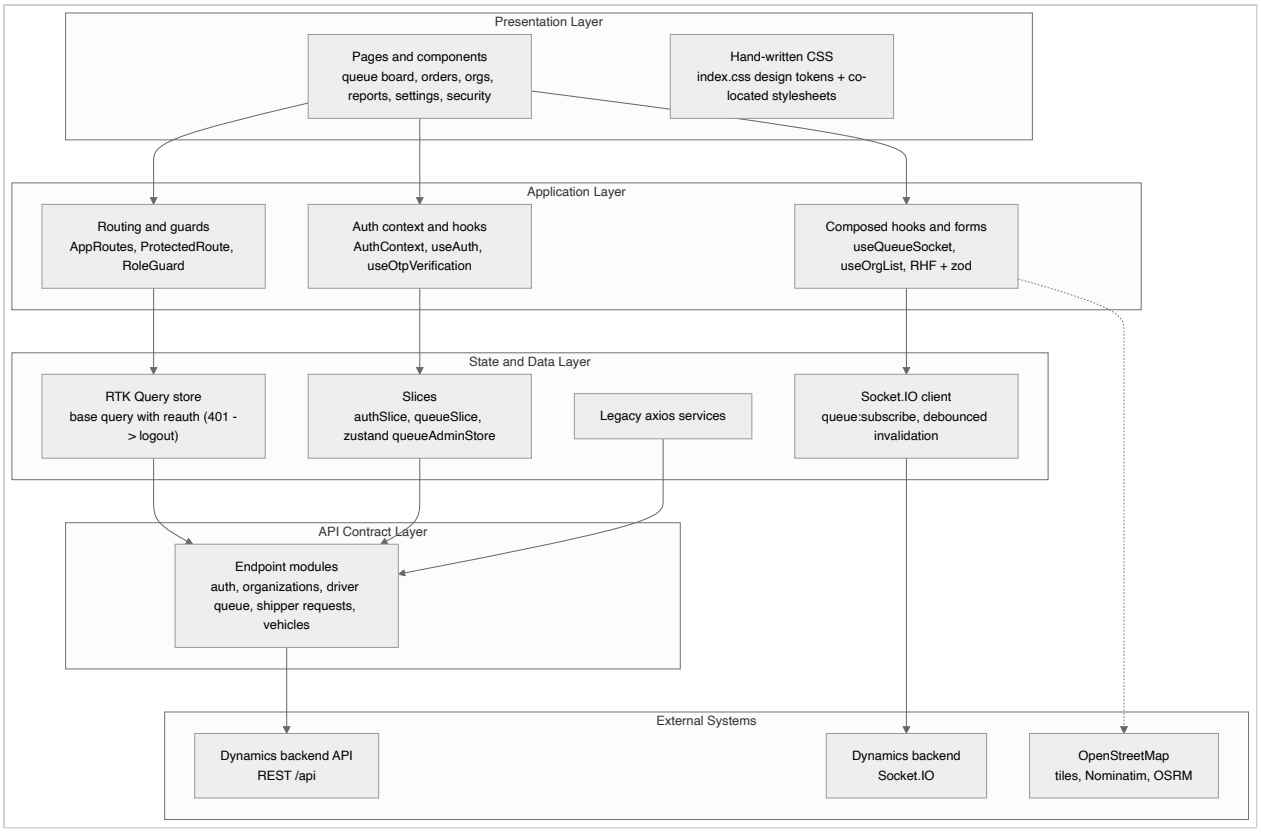


Figure 3.1. Level 2 — Architecture layers (presentation, application, state/data, API contract, external) and data paths (per `assets/l2.mmd`).

3.5 Single-page routing and the guard stack

One `BrowserRouter` is mounted in `src/main.tsx`, and every route is declared in `src/routes/AppRoutes.tsx`. Pages load through `React.lazy` behind a `Suspense` fallback (`PageLoader`), which puts each route in its own dynamic-import chunk. Two default redirects send `/` and `*` to `/dashboard`.

Authorization runs through two guards stacked on top of each other. `ProtectedRoute` first checks that there is an authenticated user (`auth.userData`), and by default that the account belongs to at least one queue organization; when there is none it sends the user to `/setup-org`. That `/setup-org` route passes `requireOrg={false}`. Sitting inside it, `RoleGuard` lets through only role ids `{11, 12, 3, 6}` and otherwise redirects to `/login`. These guards are there to keep the interface honest for whoever is using it; every request is re-checked by the backend anyway.

4. Functional decomposition

4.1 Level 1 decomposition

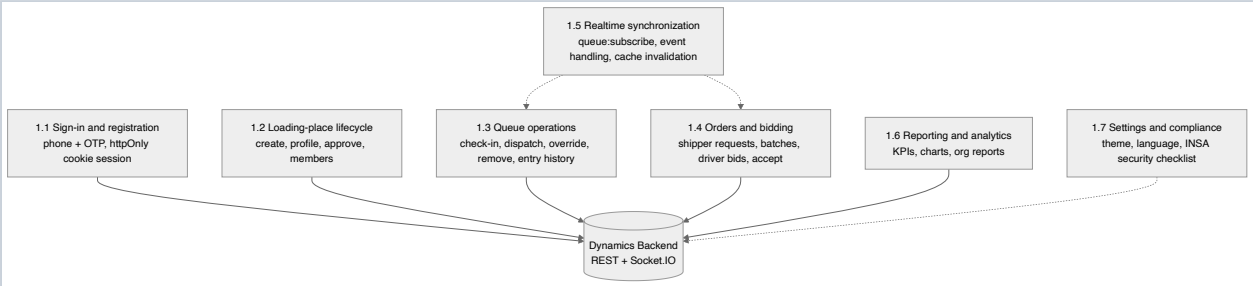


Figure 4.1. Level 1 — Functional decomposition of console capabilities (per `assets/l1.mmd`).

4.2 Key processes

Process	Description	Key modules
Sign-in and session	Phone number → OTP; the backend sets an httpOnly session cookie at `POST /api/user/verifyUserByOTP`. `AuthContext` stores the non-secret profile and connects the socket; logout calls `/api/user/logout` and clears state.	`src/components/auth/*`, `src/hooks/useOtpVerification.ts`, `src/context/AuthContext.tsx`
Loading-place approval	Create/edit the Queue Organization profile and drive the platform approval workflow (pending → approved / rejected / suspended). Admins (3/6) and queue-org admins (11) act through `/queueOrganization/:id/approve`; `queueEnabled` is toggled on approval.	`src/components/organization/SetupOrganization.tsx`, `src/pages/queue-org-manage/QueueOrgManagePage.tsx`, `src/components/queue-org-manage/*`
FIFO queue lifecycle: check-in → queue → dispatch	Operators manually check a vehicle-driver into the line (`/queue/manualCheckin`); the returned `queueNumber` places the driver in FIFO order per vehicle type. Dispatch assigns a queued entry to a shipper request (`/queue/dispatch`).	`src/components/queue/CheckinModal.tsx`, `src/components/queue/checkin/*`, `DispatchModal.tsx`, `QueueBoard.tsx`
Check-in radius / Haversine gate	A Queue Organization carries an optional `checkinRadiusKm`. The console computes great-circle distance with a Haversine implementation (`calculateDistanceKm`) for proximity cues; the authoritative geofence decision for a GPS check-in is enforced server-side.	`src/utils/formatters.ts` (`calculateDistanceKm`), `src/components/orders/bids/BidRow.tsx`
Override / remove with audit	A dispatcher repositions an entry with a reason (`PATCH /queue/entry/:queueUniqueId/override`) or removes it (`DELETE /queue/entry/:queueUniqueId`); both are audited in the backend queue audit trail.	`src/components/queue/OverrideModal.tsx`, `ConfirmCancel.tsx`, `src/lib/redux/api/driverQueueEndpoints.ts`

Process	Description	Key modules
Entry history	Per-entry change log rendered from `GET /queue/entry/:queueUniqueId/history` (column, old value, performed by, performed at).	`src/components/queue/EntryHistoryModal.tsx`
Bidding: approve → list → accept	Approve or close bidding per order (`POST /queue/bidding/approve`), list the authoritative driver bids cheapest-first (`GET /queue/bidding/order/:shipperRequestUniqueId/bids`), and accept a bid using explicit ids via `PUT /shipper/acceptDriverOffer`. A missing id fails hard and stays visible to the operator; nothing gets reconstructed by guesswork.	`src/components/orders/bids/*`, `src/components/queue/ShipperRequestsModal.tsx`, `DriverBidsModal.tsx`
Realtime subscribe / invalidate	The console subscribes to a Queue Organization room (`queue:subscribe`) and, on events, debounces and invalidates RTK Query tags (100 ms for org events, 250 ms for queue events) to trigger refetches.	`src/lib/socket.ts`, `src/hooks/useQueueSocket.ts`
Reporting	KPIs, donut/bar charts, and per-organization reports computed client-side from organization, queue-status, and shipper-request queries.	`src/pages/reports/ReportsPage.tsx`, `src/hooks/useReportsMetrics.ts`, `src/components/reports/*`
Security compliance dashboard	Renders the INSA Secure Website Management Standard content (focus areas, requirements, vulnerabilities, and a 102-item security test checklist) from `src/data/insaStandard.ts`, with locally persisted pass/fail marks.	`src/pages/security/SecurityPage.tsx`, `src/components/security/*`, `src/hooks/useSecurityChecklist.ts`, `src/utils/securityChecklist.ts`

5. Component design

Routes and guards are set up in `src/routes/AppRoutes.tsx`, with the guards themselves living in `src/components/auth/`. Pages are assembled out of feature components, and those components get their data from RTK Query hooks in `src/lib/redux/api`.

5.1 Routing and authorization

Route	Component	Guard
`/login`	`Auth` (`initialStep="login"`)	Public
`/register`	`Auth` (`initialStep="register"`)	Public
`/setup-org`	`SetupOrganization`	`ProtectedRoute requireOrg={false}`
`/organizations`	`Navigate → /dashboard`	— (redirect)
`/dashboard`	`QueueDashboardPage`	`ProtectedRoute` + `RoleGuard`
`/orgs/:queueOrganizationUniqueId`	`QueueOrgManagePage`	`ProtectedRoute + RoleGuard`
`/orgs/:queueOrganizationUniqueId/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`
`/orders`	`OrdersPage`	`ProtectedRoute + RoleGuard`
`/reports`	`ReportsPage`	`ProtectedRoute` + `RoleGuard`
`/security`	`SecurityPage`	`ProtectedRoute + RoleGuard`
`/settings`	`SettingsPage`	`ProtectedRoute` + `RoleGuard`
`/`, `*`	`Navigate → /dashboard`	— (redirect)

5.2 Feature areas

Feature	Components / Pages	Store / API
Queue board, table, cards, modals	`src/components/queue/QueueBoard.tsx`, `QueueBoardHeader.tsx`, `QueueVehicleTypeCard.tsx`, `QueueAllDriversCard.tsx`, `QueueCard.tsx`, `table/QueueDesktopTable.tsx`, `table/QueueMobileTable.tsx`, `QueueBoardModals.tsx`, `CheckinModal.tsx`, `DispatchModal.tsx`, `OverrideModal.tsx`, `ConfirmCancel.tsx`, `EntryHistoryModal.tsx`	`useGetQueueStatusQuery`, `useManualCheckinMutation`, `useDispatchQueueMutation`, `useOverrideEntryMutation`, `useRemoveEntryMutation`, `useGetEntryHistoryQuery`

Feature	Components / Pages	Store / API
Orders and bids	`src/pages/orders/OrdersPage.tsx`, `OrdersTable.tsx`, `OrdersCardsList.tsx`, `OrdersMobileCards.tsx`, `orders/bids/*` (`BidRow`, `BidsToolbar`, `BiddingBoardToggle`, `useAcceptDriverBid`), `orders/DriverBidsModal.tsx`	`useGetShipperRequestsQuery`, `useGetShipperRequestBatchesQuery`, `useGetBidsForOrderQuery`, `useApproveBiddingMutation`, `useAcceptDriverRequestMutation`
Queue organization management	`src/pages/queue-org-manage/QueueOrgManagePage.tsx`, `queue-org-manage/OrgProfileForm.tsx`, `MembersTable.tsx`, `AdminApprovalActions.tsx`, `StatusBadge.tsx`, `organization/SetupOrganization.tsx`	`useCreateQueueOrganizationMutation`, `useUpdateQueueOrganizationMutation`, `useApproveQueueOrganizationMutation`, `useListQueueOrgMembersQuery`, member lifecycle mutations
Dashboard	`src/pages/dashboard/QueueDashboardPage.tsx`, `src/components/dashboard/*` (ActiveOrgView, OrgStatusTable, OrgListToolbar, OrgListState, OrgListPagination, OrgApprovalBanner)	`useOrgList`, `useActiveOrg`, `useListQueueOrganizationsQuery`
Reports	`src/pages/reports/ReportsPage.tsx`, `src/components/reports/*` (KpiCards, DonutChart, MonthlyBarChart, OrgReportsList, OrgReportRow)	`useReportsMetrics` over organizations/queue/requests queries
Security compliance	`src/pages/security/SecurityPage.tsx`, `src/components/security/*` (SecurityScoreCards, FocusAreaAccordion, VulnerabilityTable, SecurityChecklist)	`useSecurityChecklist`, `src/data/insaStandard.ts`
Settings and layout	`src/pages/settings/SettingsPage.tsx`, `src/components/settings/*`, `src/components/layout/DashboardLayout.tsx`, `Sidebar.tsx`, `src/components/common/MobileHeader.tsx`	`useTheme`, `useAuth`, `AppRoutes`

6. Data design

6.1 Core entities and key attributes

Entity	Key attributes
User	`userId`, `userUniqueId`, `fullName`, `phoneNumber`, `email`, `roleId` (11, 12, 3, 6). Non-secret profile persisted client-side.
QueueOrganization	`queueOrganizationUniqueId`, name, type enum `customs | factory | cement | depot | mine | farm | port | other`, `latitude` / `longitude`, `checkinRadiusKm`, `approvalStatus` (`pending | approved | rejected | suspended`), `queueEnabled`, `approvedBy` / `approvedAt`.
QueueOrganizationMembership	`queueOrganizationMembershipUniqueId`, `userUniqueId`, `roleId` (11/12), `isActive`, `membershipStartDate`. Membership is the permission.
DriverQueue	`queueUniqueId`, `queueNumber`, `queueStatus` (`waiting | offered | loaded | removed | completed`), `queueRefusalCount`, `targetedShipperUserUUID`, `vehicleDriverUniqueId`, `shipperRequestUniqueId`, timestamps.
DriverQueueHistory	`historyUniqueId`, `columnName`, `oldValue`, `performedBy`, `performedAt`.
QueueAuditLog	Backend audit trail for queue mutations (check-in, dispatch, override, remove), including the override reason and the acting user; surfaced in the entry history view.
ShipperRequest	`shipperRequestUniqueId`, `shipperRequestBatchUniqueId`, `requestMode` (`individual_target | company_target`), origin/destination, `shippingCost`, `journeyStatusId`, `queueOrganizationUniqueId`, `isBiddingApproved`.
ShipperRequestBatch	`batchUniqueId`, `totalVehicles`, `requestMode`, origin/destination, `bidSummary`, aggregated bids (fetched with `includeBids=true`, `requestMode=company_target`).
DriverBid	`driverBidUniqueId`, `shipperRequestUniqueId`, `driverUserUniqueId`, `bidAmount`, `bidNotes`, `bidStatus` (`pending | selected | not_selected | withdrawn | expired`), `driverBidCreatedAt`.

6.2 Client state and storage

Store/Storage	Keys/Areas	Notes
RTK Query cache	`api` reducer with tags QueueOrganizations, QueueOrgMembers, QueueStatus, DriverQueue, DriverBids, DriverDirectory, Auth, VehicleTypes, ShipperRequests	`keepUnusedDataFor: 300`; tag-based invalidation; no focus/reconnect refetch.
Redux slices	`authSlice` (auth profile + hydration), `queueSlice` (`selectedOrgId`, `orgsCache`)	Registered in `src/lib/redux/store.ts`.
zustand store	`queueadmin:store`: persisted `selectedOrgId` plus `socketConnected` (transient)	`src/store/queueAdminStore.ts`.

Store/Storage	Keys/Areas	Notes
localStorage	`queueadmin:user` (non-secret profile), `app_dark_mode`, `app_language`	Never stores tokens; legacy `queueadmin:auth` token record is purged on read.
httpOnly cookie	Session cookie (backend-set)	Set at `POST /api/user/verifyUserByOTP`; sent with `credentials: "include"`.

6.3 Data flow (high level)

When someone acts in the UI, the handler calls an RTK Query hook or mutation, or one of the legacy axios services. The base query works out the API prefix from `VITE_API_BASE_URL` / `VITE_API_URL` (trimmed so it always ends in `/api`) and carries the httpOnly cookie with `credentials: "include"`. Whatever comes back settles into the RTK Query cache. Socket events for the queue and for organizations then invalidate tags after a short debounce, which pulls the affected queries again. Client state (the auth profile, the selected organization, theme, language, the socket flag) sits in Redux slices, zustand, React Context and localStorage; no token is ever written where JavaScript can read it.

7. Interface design

7.1 REST API

Endpoint paths are all declared in `src/utils/constant.ts`, so there is one place to change them, and the API base resolves to `<VITE_API_BASE_URL>/api`. The methods below come from the endpoint modules we actually inject. This is everything the frontend can call:

Method + path (`/api` prefix)	Purpose	Module
`POST /user/loginUser`	Request login OTP for a phone number.	auth
`POST /user/verifyUserByOTP`	Verify OTP; backend sets the session cookie.	auth
`POST /user/createUser`	Register a queue-org admin account.	auth
`POST /user/logout`	Clear the server session cookie.	auth
`GET /user/getShipperRequest4allOrSingleUser`	List shipper requests (all/single, scoped to the org).	shipper requests
`GET /queueOrganization`	List queue organizations for the user.	queue organization
`POST /queueOrganization`	Create a loading place (approvalStatus pending).	queue organization
`GET /queueOrganization/:id`	Get one organization.	queue organization
`PATCH /queueOrganization/:id`	Update the organization profile.	queue organization
`DELETE /queueOrganization/:id`	Delete (soft) the organization.	queue organization
`PATCH /queueOrganization/:id/approve`	Approve / reject / suspend; toggles `queueEnabled`.	queue organization
`GET /queueOrganization/:id/members`	List members (role 11/12, active flag).	queue organization
`POST /queueOrganization/:id/members`	Add a member (`userUniqueId` in body).	queue organization

Method + path (`/api` prefix)	Purpose	Module
`PATCH /queueOrganization/:id/members/:membershipId/deactivate`	Deactivate membership (revokes access immediately).	queue organization
`PATCH /queueOrganization/:id/members/:membershipId/reactivate`	Reactivate membership.	queue organization
`DELETE /queueOrganization/:id/members/:membershipId`	Remove membership.	queue organization
`GET /queue/status`	Queue snapshot by vehicle type for a date.	driver queue
`POST /queue/manualCheckin`	Check a vehicle-driver in; returns `queueNumber`.	driver queue
`POST /queue/dispatch`	Dispatch a queued entry to a shipper request.	driver queue
`DELETE /queue/entry/:queueUniqueId`	Remove a queue entry (audited).	driver queue
`PATCH /queue/entry/:queueUniqueId/override`	Override position with a reason (audited).	driver queue
`GET /queue/entry/:queueUniqueId/history`	Entry change history.	driver queue
`GET /queue/driverDirectory`	Search active driver/vehicle assignments for manual check-in (queue staff only).	vehicles
`POST /queue/bidding/approve`	Open/close bidding per order (`shipperRequestUniqueIds`, `approved`).	driver queue / bids
`GET /queue/bidding/order/:shipperRequestUniqueId/bids`	Authoritative driver bids for an order (cheapest-first).	driver queue / bids
`POST /shipperRequest/createRequest`	Create a shipper request / batch order.	shipper requests
`GET /shipperRequestBatch`	List batches (`company_target`, `includeBids`).	shipper requests
`PUT /shipper/acceptDriverOffer`	Accept a driver bid with explicit ids.	driver queue / bids
`GET /admin/vehicleTypes`	Reference vehicle types (falls back to built-in defaults).	vehicles

7.2 Authentication / session contract

- Primary session is an httpOnly cookie set by the backend at `POST /api/user/verifyUserByOTP`. The client never reads or stores a JWT (`getToken()` returns `null`).
- Every request sends the cookie with `credentials: "include"` (`src/lib/redux/api/base.ts`, `src/lib/api.ts`). No `Authorization` header is set.
- There is no refresh-token flow: any `401` response in the RTK Query base query dispatches `logout()` (`baseQueryWithReauth`); the axios instance clears auth and routes to `/login`.
- Session lifetime is role-based on the backend: roles 3/6 = 24h; role 11/12 and others = 365d.

7.3 Socket.IO protocol

The client connects with `withCredentials` plus an `auth` payload of `{ user, phoneNumber }`. Which identity the socket takes depends on the account role: roles 3/6 become `admin`, while queue staff (roles 11 and 12) both get the one `queueOrgAdmin` identity, because that is the key the backend fans queue events out under for either role. Subscribing happens over `queue:subscribe` / `queue:unsubscribe` against a reference-counted room keyed on `queueOrganizationUniqueId` (plus an optional `queueDate`), and the room is re-subscribed for us whenever the socket (re)connects. Queue events that arrive are read for a `messageTypes` value, while acknowledgements and ping/pong are dropped. An organization event invalidates `QueueOrganizations` after a 100 ms debounce; any other queue event invalidates `QueueStatus`, `DriverQueue` and `ShipperRequests` after a 250 ms one. If a `connect_error` comes in with a message pointing at `unauthorized` / `token`, we disconnect and the session ends.

7.4 Third-party services

Searching for an order's origin or destination goes through the Photon geocoder (`VITE_PHOTON_URL`, default `https://photon.komoot.io/api/`). Those results are merged with a built-in list of Ethiopian cities, and if the external service is unavailable we fall back to that list locally. OpenStreetMap / Nominatim / OSRM are listed as optional external map services but are not wired into this console asset.

7.5 Error-handling contract

`src/utils/parseError.ts` takes whatever failure reaches it, whether it came from axios, RTK Query, a serialized error or a plain throw, and turns it into one message a person can read. It looks for the server's `message` / `error` / `details` / `validation` fields first and only falls back to its own wording for HTTP status codes (400/401/403/404/409/429/5xx). Call sites in the UI show the result as a `sonner` toast. Two statuses are deliberately treated differently: a `401` means the session is gone and the person has to log in again, so it logs them out, while a `403` means they are signed in but not allowed to do that, so it is shown as a permission error and they stay logged in.

8. Security design

8.1 Authentication and authorization

- OTP login establishes an httpOnly cookie session; the client holds only the non-secret profile. No JWT is stored or transmitted from JavaScript.
- Route access is guarded by `ProtectedRoute` (authentication + organization presence) and `RoleGuard` (roles `{11, 12, 3, 6}`). This is a usability boundary; the backend independently enforces role and membership on every request.
- A single `401` handler in the base query logs the user out globally; a socket `401`/unauthorized handshake also ends the session and stops reconnect attempts.

8.2 Cookie session and CSRF posture

Our session is an httpOnly cookie sent with `credentials: "include"`, so CSRF protection has to come from the backend's own controls, such as same-site cookie attributes and trusted-origin checks. Those sit outside this client asset. The client does not mint its own CSRF tokens and never puts credentials where scripts can read them, and the console never writes to `document.cookie` at all.

8.3 Role enforcement boundary (client vs backend)

Concern	Client	Backend (authority)
Route visibility	Guards hide/redirect unauthorized routes.	Re-checks role/membership per request.
Bidding approval	Toggle shown only to role 11 (queue-org admin).	Enforces approval permission server-side per order.
Membership = permission	Reflects active membership in the members table.	Requires an active membership row (role 11/12) on each request.
Check-in radius	Computes Haversine distance for proximity cues.	Authoritative geofence decision for GPS check-in.

8.4 Input validation and sanitization

Every form runs on `react-hook-form` against `zod` schemas in `src/schemas/queue.ts`, covering login, registration, organization setup and profile, check-in, dispatch, order creation and override. The schemas pin down types, lengths, enums, UUIDs, coordinate ranges and quantity limits, and they carry sanitizers that strip HTML tags and control characters and collapse whitespace (`sanitizeText`, `sanitizeEmail`). React already escapes whatever it renders, which is why i18n is configured with `escapeValue: false`.

8.5 Storage hygiene and build hardening

- localStorage holds only non-sensitive values (`queueadmin:user` profile, `app_dark_mode`, `app_language`, persisted `selectedOrgId`); tokens are never stored, and legacy token records are purged on read.
- Production source maps are disabled (`sourcemap: process.env.NODE_ENV !== "production"`).

- **Honest note:** this repository does not configure a Content Security Policy or other HTTP security headers. Header/CSP hardening is a server-level responsibility of the hosting environment, which is outside the scope of this code asset; the gap is recorded in §13 and treated as deferred/mitigated at the platform boundary.

8.6 INSA compliance dashboard

The console ships a security-compliance dashboard that turns the INSA Secure Website Management Standard (Version 1.0, 2014 EC) into something a reviewer can work through as a self-assessment. All of its content comes from `src/data/insaStandard.ts`: **5 focus areas** (requirement gathering, design, implementation, hosting, operation & management), **35 requirements**, **8 common vulnerabilities** (Annex A), and a **102-item security test** checklist (Annex B). The reviewer marks each test pass or fail, the marks stay on that machine (`useSecurityChecklist`), and `src/utils/securityChecklist.ts` turns them into a score.

9. Deployment and hosting

9.1 Build

The build is a plain Vite static build: `npm run build` runs `tsc -b && vite build` and writes everything into `dist/`. Rollup pulls the heavier libraries out into chunks of their own, a `vendor` chunk (react, react-dom, react-router-dom), a `redux` chunk (@reduxjs, react-redux, zustand) and an `icons` chunk (lucide-react), while route components stay in separate dynamic-import chunks.

9.2 Environment and API base resolution

The API base is decided at compile time: `VITE_API_BASE_URL` first, then `VITE_API_URL`, then plain `/api`, with the result normalized so it always ends in `/api` (`src/utils/baseUrl.ts`). While developing, Vite proxies `/api` and `/socket.io` (WebSocket upgrade included) to `VITE_API_BASE_URL` when that is set. For the socket we prefer `VITE_WEBSOCKET_URL` / `VITE_SOCKET_URL` and otherwise use the API base or the page origin.

9.3 Hosting and SPA fallback

We serve the built `dist/` as a static site over TLS at `queue.dynamicsroute.tech`. Routing happens in the browser, so the host has to hand back the SPA entry (`index.html`) for any path it does not recognise; otherwise a deep link like `/orgs/:id` would not load. Once the app is up, its own `*` route redirects to `/dashboard`. Session cookies travel over HTTPS and carry the attributes the backend sets on them.

9.4 Environments

Env	SPA	API
Production	`https://queue.dynamicsroute.tech`	`https://app.dynamicsroute.tech`
Development	local dev (Vite, port 5173)	`https://dev.dynamicsroute.tech`

10. Cross-cutting design

- **Internationalization:** i18next/react-i18next with English and Amharic resources (`src/i18n/en.ts`, `src/i18n/am.ts`); components use inline fallback strings, and the language persists in `app_language`.
- **Theme:** light/dark mode via `ThemeContext` (`src/contexts/ThemeContext.tsx`), toggling the `dark` class on the document root and applying `src/dark.css`; preference persists in `app_dark_mode`.
- **Responsive/mobile:** dedicated mobile views and headers (`MobileHeader`, mobile queue/order cards, mobile settings) alongside desktop layouts, driven by hand-written CSS breakpoints.
- **Accessibility:** semantic elements, ARIA roles/labels on interactive controls and modals (`useModalA11y`), keyboard handling (OTP inputs, custom selects, sidebar items), and status regions.
- **Toasts:** `sonner` renders operation and error feedback at the application root.
- **Date handling:** `date-fns` plus shared formatters for queue/order dates.

> **Error boundaries**
>
> There is no React error-boundary component in this console yet, so a render error that nothing catches will not be contained inside the app. We have logged it as a known limitation (§13).

11. Design decisions and trade-offs

Decision	Rationale	Trade-off
RTK Query over react-query	One store, tag-based cache invalidation, and a single 401 reauth slot fit the queue's shared cache.	`@tanstack/react-query` remains a dead dependency and should be removed.
React Context for auth and theme	Small, low-frequency global state; avoids extra Redux plumbing.	Context re-renders; unsuitable for high-frequency data (that lives in RTK Query).
Cookie session over bearer token	Keeps the JWT out of JavaScript and removes token-handling code.	Depends on backend cookie/CSRF controls; cross-site requests need `credentials: "include"`.
Hand-written CSS	Full control, no build/utility dependency, small footprint.	More manual consistency work than a design system or utility framework.
Single socket identity for both queue roles	The backend fans queue events to the `queueOrgAdmin` key for roles 11 and 12; a separate dispatcher identity would silently receive nothing.	Client cannot distinguish role by socket identity; role checks stay per-request.
Debounced full refreshes over per-event patching	Event payloads vary and can be partial; invalidating authoritative tags is simpler and safer.	Extra network round-trips; a burst of events coalesces into one refetch.
Accept bids by explicit ids	Prevents the race where fuzzy id reconstruction silently accepts a different driver.	A missing id is a hard error the operator must resolve by reloading bids.

12. Requirements-to-design traceability

This table maps the SRS requirement families (FR-*/NFR-*) onto the design elements that answer them. The main ones are below:

SRS family	Design reference	Notes
FR-AUTH	`useOtpVerification`, `AuthContext`, `services/auth.service.ts`, `base.ts`	OTP login, cookie session, global 401 logout, socket 401 session end.
FR-ROLE/ACCESS	`ProtectedRoute.tsx`, `RoleGuard.tsx`	Roles {11, 12, 3, 6}; organization-presence requirement.
FR-ORG	`SetupOrganization`, `QueueOrgManagePage`, `OrgProfileForm`, membership mutations	Create/edit profile, approval workflow, membership lifecycle.
FR-QUEUE	`QueueBoard` and queue components, `driverQueueEndpoints.ts`	Check-in → queue → dispatch, override/remove, entry history, FIFO number.
FR-BID	`src/components/orders/bids/*`, `DriverBidsModal`, `approveBidding` / `getBidsForOrder` / `acceptDriverRequest`	Approve → list → accept via explicit ids.
FR-ORDER	`OrdersPage`, `CreateOrderModal`, shipper request endpoints	List/view shipper requests and batches; create request.
FR-REALTIME	`src/lib/socket.ts`, `useQueueSocket`	Subscribe/unsubscribe and debounced tag invalidation.
FR-REPORT	`ReportsPage`, `useReportsMetrics`	Client-side KPIs, charts, and per-org reports.
NFR-SEC	`parseError`, zod schemas, `lib/auth.ts`, `src/data/insaStandard.ts`	Input validation, storage hygiene, INSA compliance self-assessment.
NFR-DEPLOY	`vite.config.ts`, `baseUrl.ts`, hosting at `queue.dynamicsroute.tech`	Env-driven base URL, dev proxy, static SPA fallback.

13. Known limitations and future work

Item	Status	Planned action
No refresh-token flow	NOTED	Any expiration ends the session (401 → logout); add refresh only if the backend offers it.
Settings: profile / notifications / 2FA disabled	NOTED	No profile PATCH and no notification/2FA backend; controls are intentionally disabled rather than faked.
Dashboard counts computed client-side	NOTED	Metrics are derived from fetched lists (limit 100) in the browser; move to server aggregates for scale.
Socket payloads trigger refetch only	BY DESIGN	Events invalidate tags; no per-event cache patching is applied.
Dead `@tanstack/react-query` dependency	OPEN	Nothing imports it; remove from `package.json`.
Unreachable `/organizations` page	OPEN	The route redirects to `/dashboard`; the standalone page is not reachable and should be deleted or re-wired.
No CSP or HTTP security headers in this repo	DEFERRED	Server-level control, outside this code asset; mitigate at hosting/reverse-proxy layer.
Optional third-party map use	OPTIONAL	Only Photon place search is wired; OSM/Nominatim/OSRM remain optional.
No in-app error boundary	OPEN	Add a top-level React error boundary to contain render errors.

End of document

That closes out the System & Architecture Design Document (SDD-1.0.0) for Dynamics Queue Admin, submitted by Dynamics Transport. Everything above was checked against the repository as it stands in `queadmin-frontend` version 1.0.0, so what you read here is what is built.